

---

Oliver Krauss

# Relational Databases & SQL

RDB | MTD.ba VZ SS26

Hagenberg 10.03.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY  
OF APPLIED SCIENCES  
UPPER AUSTRIA

# Data Management

Many applications in software and web require **data management**:

- **Structured data** needed in application
- Store it in a **persistent** manner (retain data after shutdown or crashes, make sure it is correct)
- **Efficiently query** information within large amounts of information
- **Update** data while upholding **constraints**
- Enable **multiple users** to write and read data **concurrently**

# Example: Student Management System

Access from custom software (Levis), a WebSite (Moodle), ... Requirements:

- **Structured data** representing students, grades, class assignments etc.
- **Exception handling** when these applications shutdown or crash
- **Efficiently query** which students are assigned to which classes over several years of history
- **Update** new enrolling students or dropouts
- **multiple students** want to view the same course at the same time **concurrently**

# Naive implementation

- Implement data structures in a programming language (Java, PHP, ...)
- Define ad-hoc file format to store data on disc
- Add synchronization mechanism for failure management
- Store data in memory and implement sorting algorithms and binary search trees to query data efficiently
- Update functionality with code checking on the fly to make sure no business constraints are violated
- Define different user roles (student, teacher, admin,...), authentication mechanism
- Make sure no race conditions occur on the data

# Naive implementation

- Implement data structures in a programming language (Java, PHP, ...)
- Define ad-hoc file format to store data on disc
- Add synchronization mechanism for failure management
- Store data in memory and implement sorting algorithms and binary search trees to query data efficiently
- Update functionality with code checking on the fly to make sure no business constraints are violated
- Define different user roles (student, teacher, admin,...), authentication mechanism
- Make sure no race conditions occur on the data

**Lots of unnecessary work**, must be done for every application that accessess the data!

# Database Management Systems (DBMS)

## Database Management Systems (DBMS)

Is a software that **simplifies data management** of applications by handling **data access**, concurrency, business rules, querying, ...

## Database

Is a collection of data managed by a DBMS. A DBMS can usually handle multiple databases

# Tables

| <b>StudentID</b> | <b>Enrolment</b> | <b>Name</b>  | <b>Avg. Grade</b> | <b>Department</b> |
|------------------|------------------|--------------|-------------------|-------------------|
| 1001             | 2025-08-31       | Marty McFly  | 1.0               | MTD               |
| 1002             | 2025-09-02       | Dorothy Gale | 2.4               | MTD               |
| 1003             | 2025-09-05       | Neo Anderson | 3.5               | IM                |

# Tables

## Table

| <b>StudentID</b> | <b>Enrolment</b> | <b>Name</b>  | <b>Avg. Grade</b> | <b>Department</b> |
|------------------|------------------|--------------|-------------------|-------------------|
| 1001             | 2025-08-31       | Marty McFly  | 1.0               | MTD               |
| 1002             | 2025-09-02       | Dorothy Gale | 2.4               | MTD               |
| 1003             | 2025-09-05       | Neo Anderson | 3.5               | IM                |

# Tables

**Record**  
**Row**  
**Tuple**

| <b>StudentID</b> | <b>Enrolment</b> | <b>Name</b>  | <b>Avg. Grade</b> | <b>Department</b> |
|------------------|------------------|--------------|-------------------|-------------------|
| 1001             | 2025-08-31       | Marty McFly  | 1.0               | MTD               |
| 1002             | 2025-09-02       | Dorothy Gale | 2.4               | MTD               |
| 1003             | 2025-09-05       | Neo Anderson | 3.5               | IM                |

# Tables

## Column / Field / Attribute

| StudentID | Enrolment  | Name         | Avg. Grade | Department |
|-----------|------------|--------------|------------|------------|
| 1001      | 2025-08-31 | Marty McFly  | 1.0        | MTD        |
| 1002      | 2025-09-02 | Dorothy Gale | 2.4        | MTD        |
| 1003      | 2025-09-05 | Neo Anderson | 3.5        | IM         |

**Record**  
**Row**  
**Tuple**

# Tables

## Column / Field / Attribute

| StudentID | Enrolment  | Name         | Avg. Grade | Department |
|-----------|------------|--------------|------------|------------|
| 1001      | 2025-08-31 | Marty McFly  | 1.0        | MTD        |
| 1002      | 2025-09-02 | Dorothy Gale | 2.4        | MTD        |
| 1003      | 2025-09-05 | Neo Anderson | 3.5        | IM         |

**Record**  
**Row**  
**Tuple**

**Column Value**

# Tables

- A relational database consists of **tables**
- Each table consists of **records**
- Each record consists of **column values** corresponding to **columns**

# Row Identities and Relationships

| <b>StudentID</b> | <b>Enrolment</b> | <b>Name</b>  | <b>Avg. Grade</b> | <b>Department</b> |
|------------------|------------------|--------------|-------------------|-------------------|
| 1001             | 2025-08-31       | Marty McFly  | 1.0               | MTD               |
| 1002             | 2025-09-02       | Dorothy Gale | 2.4               | MTD               |
| 1003             | 2025-09-05       | Neo Anderson | 3.5               | IM                |

# Row Identities and Relationships

## Primary Key

| <b>StudentID</b> | <b>Enrolment</b> | <b>Name</b>  | <b>Avg. Grade</b> | <b>Department</b> |
|------------------|------------------|--------------|-------------------|-------------------|
| 1001             | 2025-08-31       | Marty McFly  | 1.0               | MTD               |
| 1002             | 2025-09-02       | Dorothy Gale | 2.4               | MTD               |
| 1003             | 2025-09-05       | Neo Anderson | 3.5               | IM                |

# Row Identities and Relationships

## Primary Key

## Foreign Key

| <b>StudentID</b> | <b>Enrolment</b> | <b>Name</b>  | <b>Avg. Grade</b> | <b>Department</b> |
|------------------|------------------|--------------|-------------------|-------------------|
| 1001             | 2025-08-31       | Marty McFly  | 1.0               | MTD               |
| 1002             | 2025-09-02       | Dorothy Gale | 2.4               | MTD               |
| 1003             | 2025-09-05       | Neo Anderson | 3.5               | IM                |

# Row Identities and Relationships

## Primary Key

| StudentID | Enrolment  | Name         | Avg. Grade | Department |
|-----------|------------|--------------|------------|------------|
| 1001      | 2025-08-31 | Marty McFly  | 1.0        | MTD        |
| 1002      | 2025-09-02 | Dorothy Gale | 2.4        | MTD        |
| 1003      | 2025-09-05 | Neo Anderson | 3.5        | IM         |

## Foreign Key

## Primary Key

| Department | Name                    |
|------------|-------------------------|
| MTD        | Medientechnik & -design |
| IM         | Interactive Media       |
| DA         | Digital Arts            |



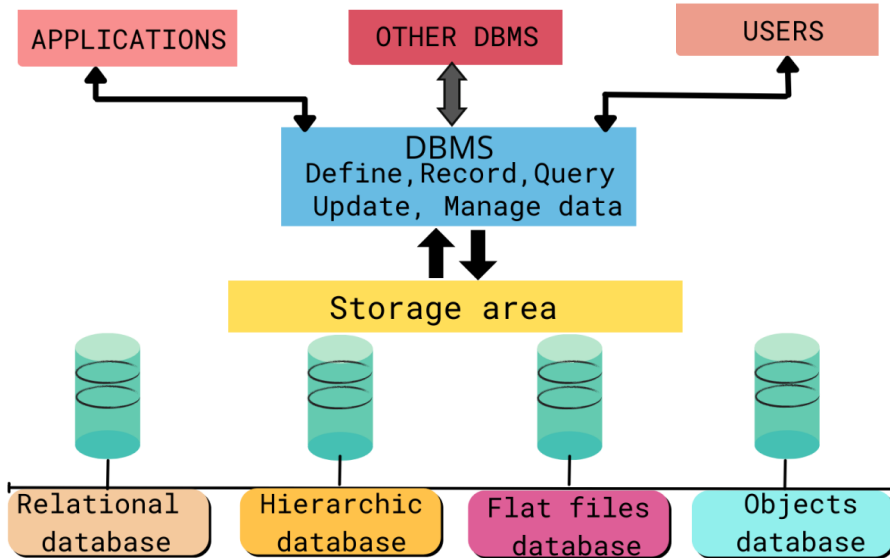
# Tables

- Each record is **uniquely** identified by a **primary key**
- A primary key consists of **one or more columns**
- A table can relate to another table via a **foreign key**
- The foreign key corresponds to the **primary key** in another table
- I.e. a foreign key consists of **one or more columns**

# Tables

- Each record is **uniquely** identified by a **primary key**
- A primary key consists of **one or more columns**
- A table can relate to another table via a **foreign key**
- The foreign key corresponds to the **primary key** in another table
- I.e. a foreign key consists of **one or more columns**

# DATABASE MANAGEMENT SYSTEM



# Structured Query Language (SQL)

- Is an ANSI/ISO standard since 1986/87
  - But different dialects exist!
  - Not every database works the same
- Is a declarative language
  - Describes desired result
  - Does not describe the computations
- used to create, query, and modify databases

# SQL - Components I

## Data Definition Language (DDL)

Defines **database schema**, i.e. tables, including modifying and deleting.

## Data Manipulation Language (DML)

Commands that **manipulate data**, i.e. inserting, updating or deleting records.

## Data Query Language (DQL)

For **querying** the data from the database. Only one command → SELECT.

# SQL - Components II

## Data Control Language (DCL)

Grant or revoke **privileges** to users, i.e. allow/deny users to run commands on specific tables.

## Transaction Control Language (TCL)

Used to group commands, and making sure your **data stays consistent**.

# SQL Command Overview

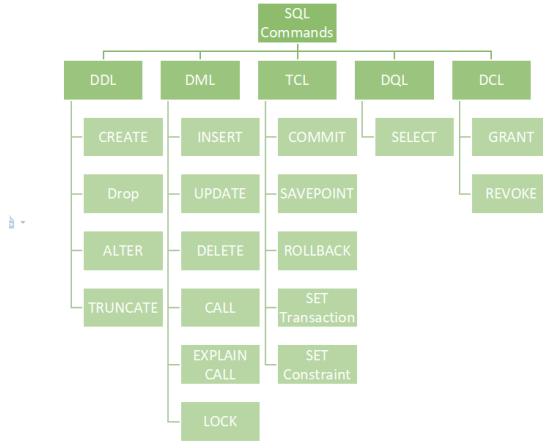


Figure: Source: <https://www.geeksforgeeks.org/sql-ddl-dql-dml-dcl-tcl-commands/>

# Data Definition Language (DDL)

## Data Definition Language (DDL)

Defines **database schema**, i.e. tables, including modifying and deleting.

# Create Database

Command:

```
1      CREATE DATABASE database_name;
```

Example:

```
1      CREATE DATABASE Hypermedia;
```

# Create Table

Command:

```
1      CREATE TABLE table_name
2      (
3      Column1 datatype (size),
4      column2 datatype (size),
5      ...
6      ...
7      columnN datatype(size)
8      );
```

Example:

```
1      CREATE TABLE Students
2      (
3      StudentID INT,
4      Enrolment DATE,
5      Name VARCHAR(100),
6      Avg_Grade NUMERIC(1,2),
7      Department VARCHAR(3)
8      );
```

# Data Types

```
1 CREATE TABLE Students
2 (
3   StudentID INT ,
4   Enrolment DATE,
5   Name VARCHAR(100) ,
6   Avg_Grade NUMERIC(1,2) ,
7   Department VARCHAR(3)
8 );
```

data type

size limited to 100 characters

size limited to 1 number (0-9) and 2 points after comma (e.x. 2.31)



Data types depend on the specific database you use, every database has (some) specific ones. See [https://www.w3schools.com/sql/sql\\_datatypes.asp](https://www.w3schools.com/sql/sql_datatypes.asp)

# Dates

```
1 CREATE TABLE Students
2 (
3 StudentID INT,
4 Enrolment DATE ,
5 Name VARCHAR(100),
6 Avg_Grade NUMERIC(1,2),
7 Department VARCHAR(3)
8 );
```

## DATE data type

- DATE - YYYY-MM-DD
- DATETIME - YYYY-MM-DD HH:MI:SS
- YEAR - YYYY
- ...



Date formats slightly differ in databases.

# Constraints - PRIMARY KEY

```
1 CREATE TABLE Students
2 (
3 StudentID INT PRIMARY KEY ,
4 Enrolment DATE,
5 Name VARCHAR(100),
6 Avg_Grade NUMERIC(1,2),
7 Department VARCHAR(3)L
8 );
```

## **PRIMARY KEY** constraint:

This key uniquely identifies each record in the table.

The key can be referenced as *foreign key* in other tables.

Only ONE primary key can exist, but it can contain of multiple columns

# Constraints - PRIMARY KEY over multiple columns

```
1  CREATE TABLE Students
2  (
3  SId INT,
4  SYear YEAR,
5  Enrolment DATE,
6  Name VARCHAR(100),
7  Avg_Grade NUMERIC(1,2),
8  Department VARCHAR(3),
9  PRIMARY KEY (SId,SYear,Department))
10 );
```

**PRIMARY KEY** over multiple columns:

The combination of all three columns is the primary key of a record.

E.g. SId = 001, SYear = 2025, Department = MTD; Primary Key = "0012025MTD"

In some databases the Constraint keyword must be used: **CONSTRAINT PK\_ID**

**PRIMARY KEY (SId,SYear,Department)**

# Constraints - NOT NULL

```
1 CREATE TABLE Students
2 (
3 StudentID INT PRIMARY KEY,
4 Enrolment DATE NOT NULL ,
5 Name VARCHAR(100) NOT NULL ,
6 Avg_Grade NUMERIC(1,2),
7 Department VARCHAR(3) NOT NULL
8 );
```

## **NOT NULL** constraint:

Value **MUST** always exist (on INSERT and UPDATE)

Not needed for primary key as the primary key can never be null!



In most databases an empty string " " is valid in this constraint.

# Constraints - UNIQUE

```
1  CREATE TABLE Students
2  (
3  StudentID INT PRIMARY KEY,
4  Enrolment DATE NOT NULL,
5  Name VARCHAR(100) NOT NULL UNIQUE ,
6  Avg_Grade NUMERIC(1,2),
7  Department VARCHAR(3) NOT NULL
8  );
```

## **UNIQUE** constraint:

Value can only occur once in this column.  
Not needed for primary key as the  
primary key are always unique!

# Constraints - UNIQUE over multiple columns

```
1  CREATE TABLE Students
2  (
3  StudentID INT PRIMARY KEY,
4  Enrolment DATE NOT NULL,
5  FirstName VARCHAR(100) NOT NULL,
6  LastName VARCHAR(100) NOT NULL
7  Avg_Grade NUMERIC(1,2),
8  Department VARCHAR(3) NOT NULL,
9  CONSTRAINT UC_NAME
10     UNIQUE (FirstName,LastName)
11 );
```

## **UNIQUE** on multiple columns:

Value can only occur once over all columns.  
E.g. "Jackie Chan" and "Jackie Cooper" are allowed, since only the combination of First and Last Name must be unique.

# Auto Increment

```
1 CREATE TABLE Students
2 (
3 StudentID INT
4 PRIMARY KEY AUTO_INCREMENT ,
5 Enrolment DATE NOT NULL,
6 Name VARCHAR(100) NOT NULL UNIQUE,
7 Avg_Grade NUMERIC(1,2),
8 Department VARCHAR(3) NOT NULL
9 );
```

## **AUTO INCREMENT:**

Numbers are generated automatically when a new record is inserted. Usually starts with 1 but can be modified with **AUTO\_INCREMENT=100**

# Foreign Keys

```
1  CREATE TABLE Students
2  (
3  StudentID INT
4      PRIMARY KEY AUTO_INCREMENT,
5  Enrolment DATE NOT NULL,
6  Name VARCHAR(100) NOT NULL UNIQUE,
7  Avg_Grade NUMERIC(1,2),
8  Department VARCHAR(3) NOT NULL
9      FOREIGN KEY
10         REFERENCES Departments(
11             Department)
12 );
```

## FOREIGN KEY:

References another table.

Unlike primary key this is allowed to be null (no reference set).

# Data Manipulation Language (DML)

## Data Manipulation Language (DML)

Commands that **manipulate data**, i.e. inserting, updating or deleting records.

Together with the Data Query Language (DQL) it relates to the CRUD principle:

**C**reate - **INSERT INTO**

**R**ead - **SELECT**

**U**pdate - **UPDATE**

**D**elele - **DELETE**

# Add new Records to Database - INSERT

Example:

Command:

```
1  INSERT INTO table_name
2      (column1, column2, ...)
3  VALUES
4      (value1, value2, ...);
```

```
1  INSERT INTO Students
2      (StudentID, Enrolment, Name,
3      Avg_Grade, Department)
4  VALUES
5      (101, "2025-03-01", "Jade Chan"
6      , 1.2, "MTD")
```

# Add new Records to Database - INSERT

```
1  INSERT INTO Students
2    (StudentID, Enrolment, Name,
3     Avg_Grade, Department)
4  VALUES
5    (101, "2025-03-01", "Jade Chan"
6     , 1.2, "MTD");
```

If all columns are filled this is optional. E.g.  
**INSERT INTO Students VALUES ...**

Only a subset of columns can be filled. E.g.  
**INSERT INTO Students (StudentID, Name, Department)) ...**

Multiple records can be added with "," separation. E.g. ... **VALUES (record1), (record2), ... (recordN);**

# Modify existing Records - UPDATE

Command:

```
1  UPDATE table_name
2  SET column1 = value1, column2 =
    value2, ...
3  WHERE condition;
```

Example:

```
1  UPDATE Students
2  SET Name = 'Stacy Chan',
    Department = "IM"
3  WHERE StudentID=101;
```

# Modify existing Records - UPDATE

```
1  UPDATE Students
2  SET Name = 'Stacy Chan',
    Department = "IM"
3  WHERE StudentID=101;
```

All records in the table will be updated that apply to that condition.

For example ... **WHERE Department="MTD"** would update every student in this department.



Warning! WHERE is optional. If you omit the WHERE condition ALL records will be updated.

# Delete records - DELETE

Command:

```
1  DELETE FROM table_name WHERE  
    condition;
```

Example:

```
1  DELETE FROM Students WHERE  
    StudentID=101;
```

# Delete records - DELETE

```
1  DELETE FROM Students WHERE  
    StudentID=101;
```

All records in the table will be deleted that apply to that condition.  
For example ... **WHERE Department="MTD"** would delete every student in this department.



Warning! WHERE is optional. If you omit the WHERE condition ALL records in the table will be deleted.